



Capítulo 7

Automatización de APIs con RequestsLibrary

Objetivos:

- Describir los métodos HTTP principales (GET, POST, PUT, DELETE) y sus diferencias semánticas en el contexto REST.
- Explicar el propósito de los códigos de estado HTTP y cómo interpretarlos durante la automatización de pruebas.
- Interpretar la estructura de una solicitud HTTP — headers y payload— para diseñar pruebas de API con Robot Framework.
- HTTP es la base de REST: cada operación se expresa como un método (GET, POST, PUT, DELETE) aplicado a una URL que representa un recurso.

7.1: REST/HTTP: Métodos, Códigos de Estado, Headers y Payloads

- Introducción

REST organiza recursos como URLs operadas con verbos HTTP:

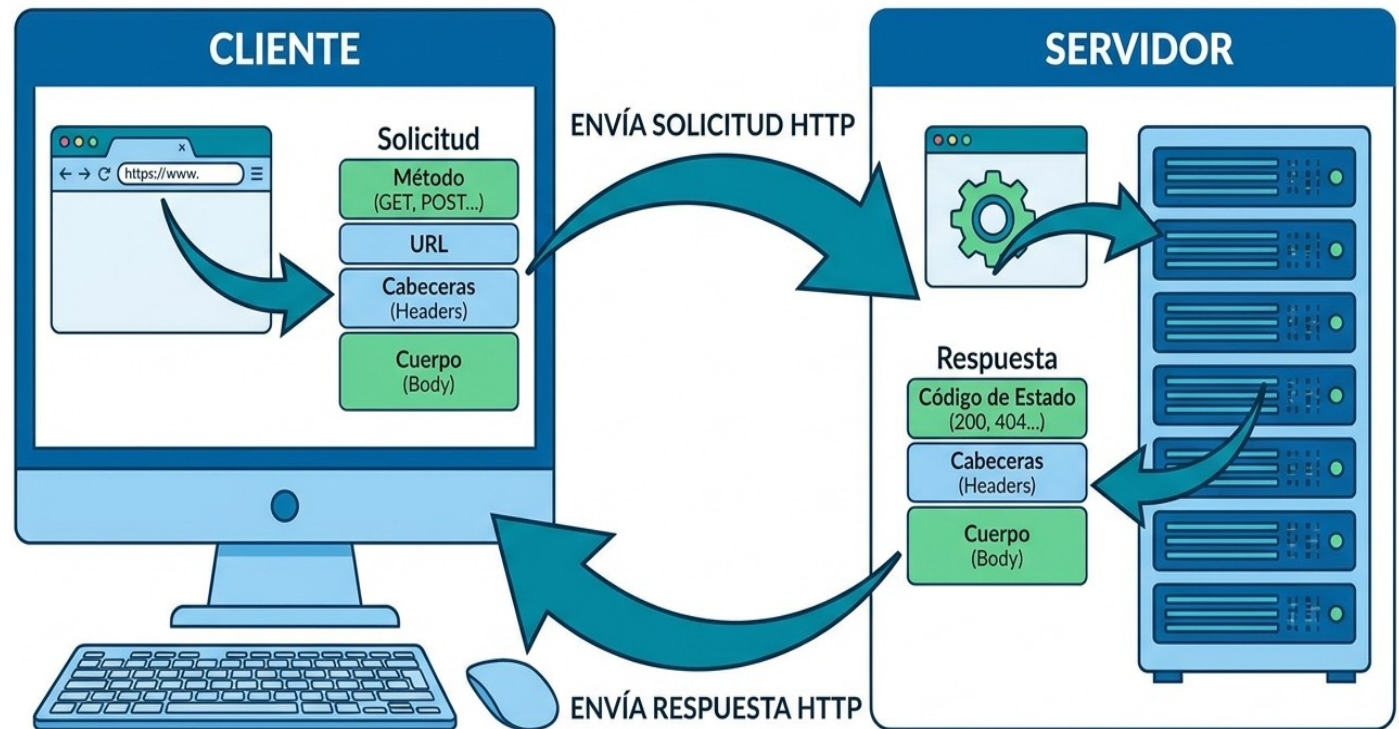
GET (leer), POST (crear), PUT (reemplazar), DELETE (eliminar).

Los códigos de estado (2xx/4xx/5xx) comunican el resultado.

HTTP y REST: Conceptos Fundamentales

- ▶ REST organiza recursos como URLs operadas con verbos HTTP.
- ▶ El cliente envía una solicitud; el servidor responde con código y datos.
- ▶ GET recupera, POST crea, PUT reemplaza, DELETE elimina.
- ▶ Idempotente: repetir la solicitud produce el mismo resultado.

DIAGRAMA DEL CICLO DE SOLICITUD-RESPUESTA HTTP

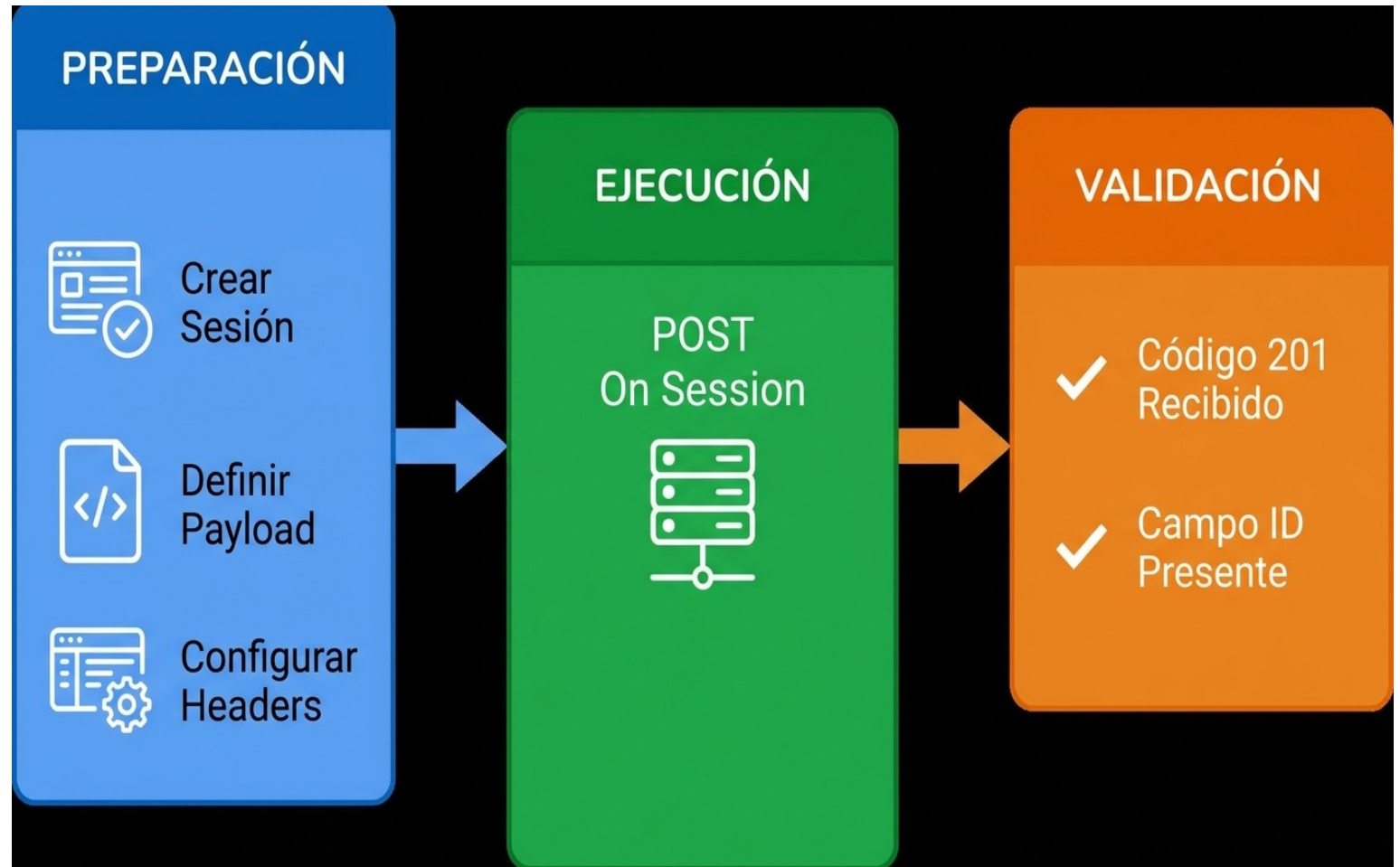


Métodos HTTP: Semántica y Propiedades

Método	Semántica	¿Envía body?	¿Idempotente?
GET	Obtener un recurso o colección	No	Sí
POST	Crear un nuevo recurso	Sí	No
PUT	Reemplazar completamente un recurso	Sí	Sí
DELETE	Eliminar un recurso	Opcional	Sí

Prueba POST con RequestsLibrary

- ▶ Create Session establece la conexión reutilizable al servidor.
- ▶ El payload se define como diccionario; RF lo serializa a JSON.
- ▶ Should Be Equal valida el código de estado 201 Created.
- ▶ jsonplaceholder.typicode.com es ideal para practicar sin servidor propio.



7.2: Autenticación: Bearer Tokens, Sesiones Persistentes y Datos Dinámicos

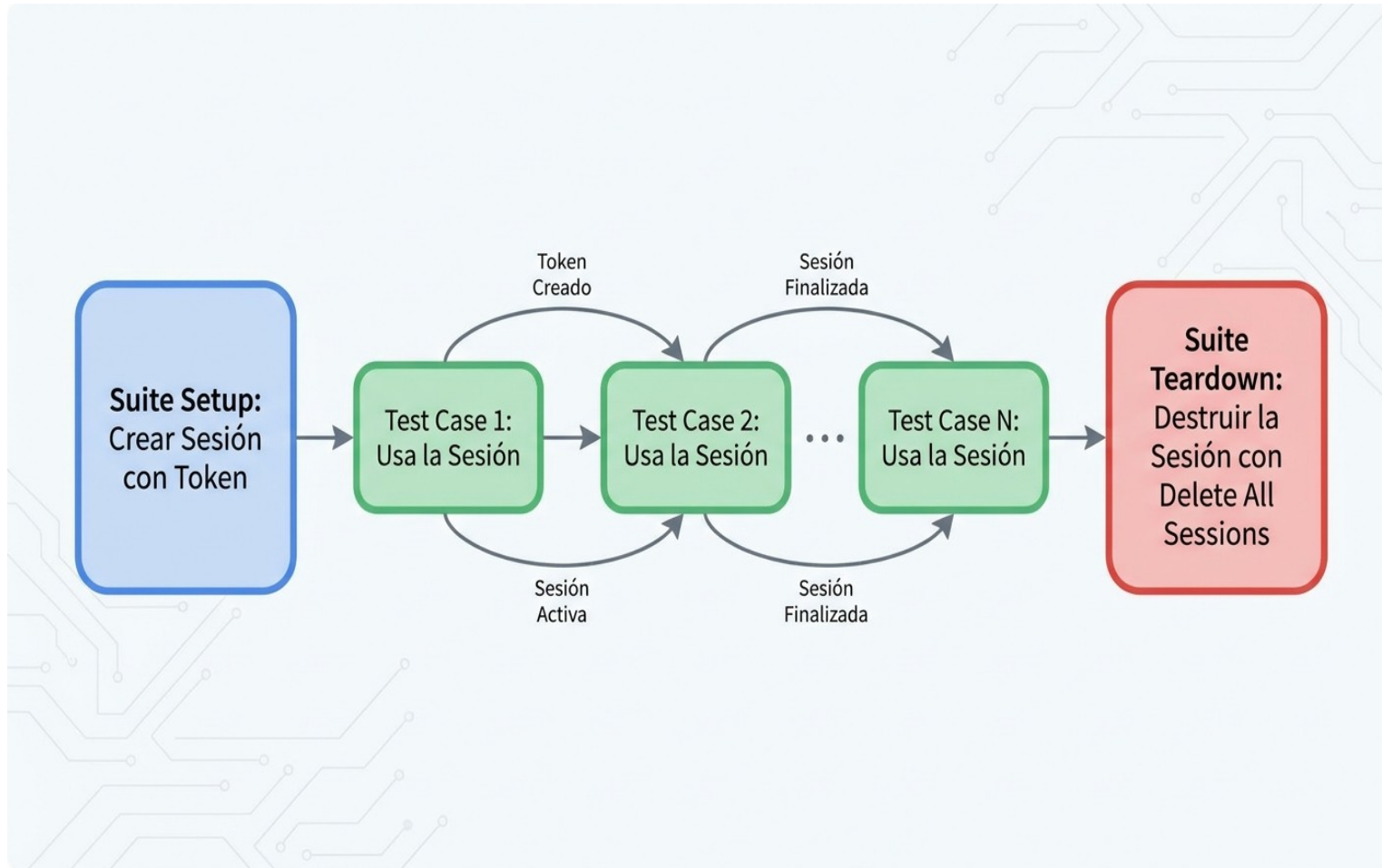
- Introducción

El esquema Bearer Token transmite la credencial en el header Authorization. Create Session mantiene configuración persistente entre peticiones de toda la suite.

Parámetros Clave deCreate Session

Parámetro	Descripción
alias	Nombre interno para referenciar la sesión
url	URL base del servidor (sin ruta)
headers	Diccionario de encabezados por defecto (incluye el Bearer token)
verify	Verificación de certificado SSL (True/False)
timeout	Tiempo máximo de espera en segundos

Sesiones Persistentes: Ciclo de Vida



- ▶ La sesión conserva headers, cookies y SSL entre peticiones.
- ▶ Suite Setup crea la sesión; Suite Teardown la destruye.
- ▶ El alias es un identificador global compartido por todos los tests.
- ▶ Un solo login por suite elimina overhead de re-autenticación.

7.3: Validación JSON: JSONPath, Aserciones de Esquema y Contract Testing Básico

- Introducción

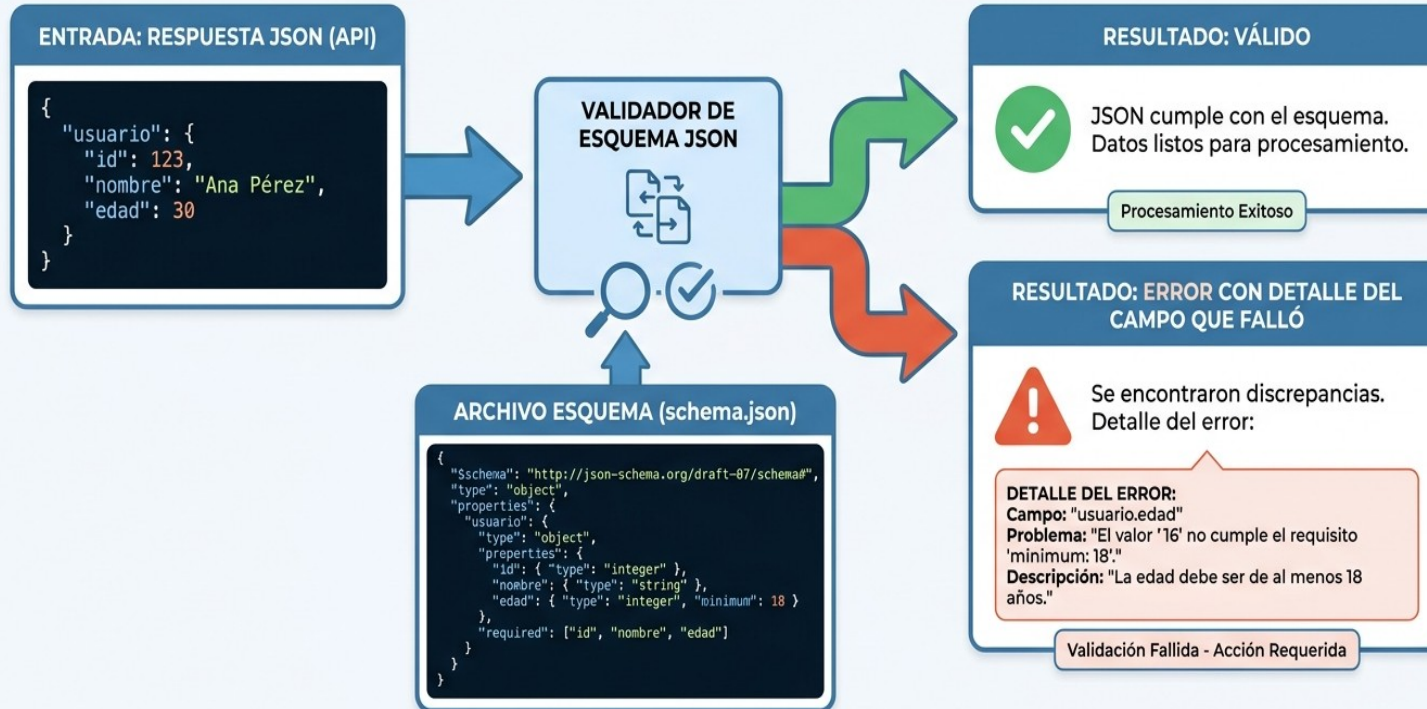
La sintaxis extendida de RF accede a valores anidados del JSON sin librerías adicionales. Contract testing valida estructura, no solo valores puntuales.

JSONPath: Concepto y Sintaxis Esencial

- ▶ JSONPath es un lenguaje de consulta para JSON, análogo a XPath para XML.
- ▶ La raíz del documento se representa con \$; los campos se navegan con ..
- ▶ \$.usuario.email accede a campos anidados; \$.items[*].id extrae todos los IDs.
- ▶ \$..estado busca recursivamente en cualquier nivel del documento.
- ▶ En Robot Framework se usa JSONLibrary con la keyword Get Value From Json.
- ▶ Get Value From Json retorna siempre una lista; usar [0] para un único valor.

Aserciones de Esquema: Verificar la Estructura

DIAGRAMA DE FLUJO: VALIDACIÓN DE RESPUESTA JSON CONTRA UN ESQUEMA

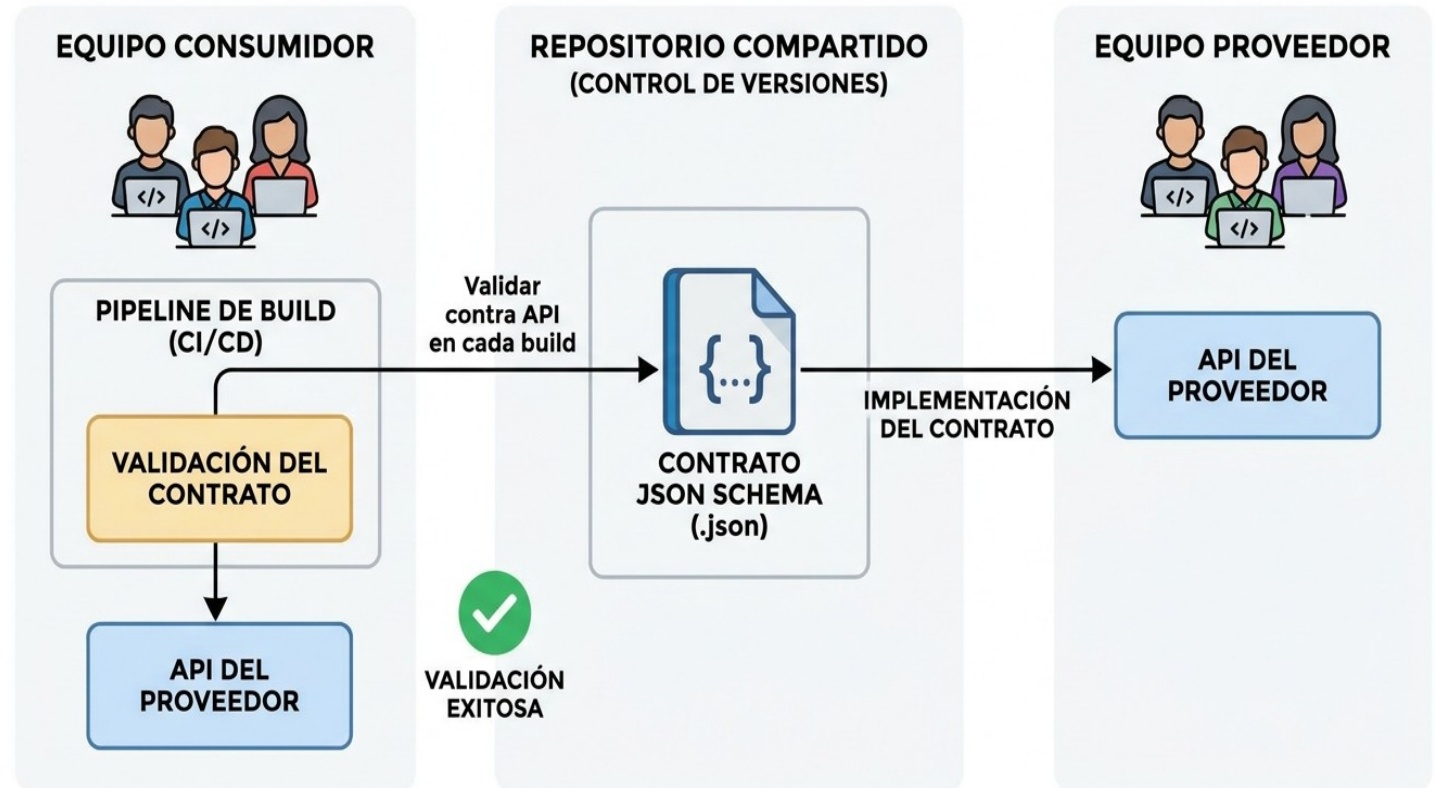


- ▶ Verifica campos obligatorios, tipos de datos y ausencia de campos inesperados.
- ▶ Valida la estructura completa, no valores puntuales como JSONPath.
- ▶ El estándar es JSON Schema; en Robot Framework se usa Validate Json By Schema File.
- ▶ Actúa como red de seguridad en regresión ante cambios estructurales no anunciados.

Contract Testing: Estabilidad Entre Servicios

- ▶ Detecta cambios incompatibles en la API antes de llegar a producción.
- ▶ Los contratos se versionan en el repositorio del equipo consumidor.
- ▶ Tag contrato permite ejecutarlos de forma aislada: `robot --include contrato`.
- ▶ Actualizar el contrato es una decisión explícita, no una sorpresa.

DIAGRAMA DE VALIDACIÓN DE CONTRATOS CON JSON SCHEMA



7.4: Diseño de Suites API: Estructura Smoke vs. Regresión y Organización por Endpoint

- Introducción

Suite smoke: pocos casos críticos, cada despliegue, rápida.

Suite regresión: cobertura completa, antes de release.

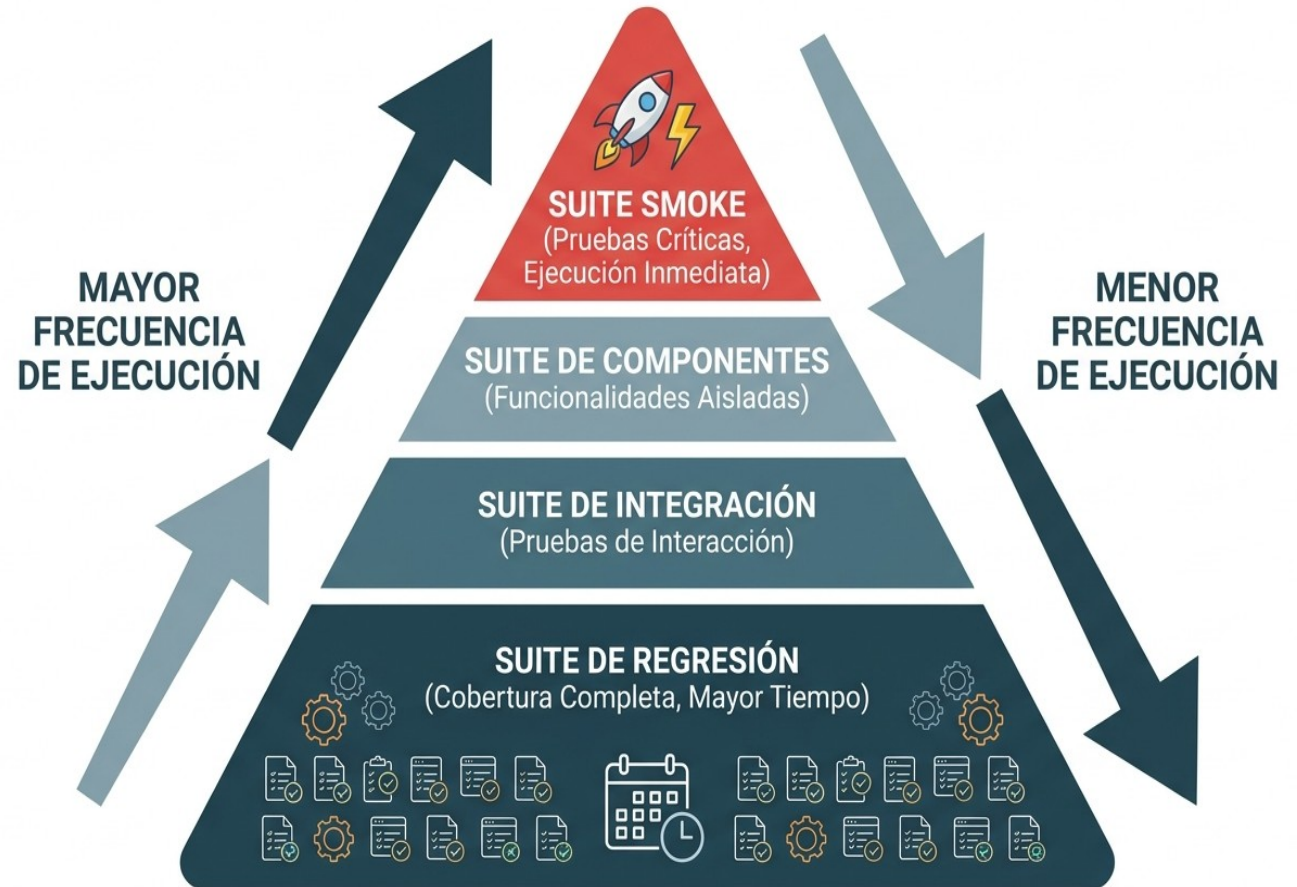
expected_status=any habilita pruebas negativas deliberadas.

Suites Smoke vs. Regresión: Concepto Central

- ▶ Una suite agrupa casos de prueba bajo un propósito común dentro del proyecto.
- ▶ Suite smoke: pocas pruebas, rápidas, verifican que el sistema 'respira' (signos vitales).
- ▶ Suite regresión: amplia cobertura funcional, escenarios positivos, negativos y límite.
- ▶ Analogía médica: primero pulso y respiración (smoke), luego análisis completos (regresión).
- ▶ Si falla una prueba smoke, continuar con regresión carece de sentido.
- ▶ Ambas suites son complementarias; se ejecutan en momentos distintos del ciclo de vida.

Flujo Smoke → Regresión en el Pipeline CI/CD

- ▶ Smoke se ejecuta primero en cada despliegue automático.
- ▶ Solo si smoke pasa, el pipeline lanza la regresión completa.
- ▶ Esto ahorra tiempo y recursos en ciclos de integración.
- ▶ El orden garantiza feedback rápido ante fallos críticos.



expected_status=any: Pruebas Negativas sin Interrupciones

- ▶ Por defecto, RequestsLibrary lanza excepción al recibir respuestas 4xx o 5xx.
- ▶ El parámetro expected_status=any desactiva ese comportamiento automático.
- ▶ Permite que el test case valide explícitamente el código de error esperado.
- ▶ Es esencial para pruebas de escenarios negativos y validación de errores.
- ▶ Sin él, un 404 esperado haría fallar el test antes de poder verificarlo.
- ▶ Patrón recomendado: usar expected_status=any + Should Be Equal As Integers.

Resumen del capítulo

- **HTTP es la base de REST:** cada operación se expresa como un método (GET, POST, PUT, DELETE) aplicado a una URL que representa un recurso.
- **Los métodos tienen semántica precisa:** GET recupera, POST crea, PUT reemplaza, DELETE elimina. Respetar esta semántica facilita la lectura y el mantenimiento de las pruebas.
- **Los códigos de estado son la primera validación:** un 200 o 201 indica éxito; un 4xx apunta a un error del cliente; un 5xx señala un problema en el servidor.
- **Los headers contextualizan la comunicación:** Content-Type y Authorization son los más relevantes en la automatización de APIs.
- **El payload es el contenido:** en APIs REST modernas suele ser JSON; debe estar correctamente formateado y alineado con el Content-Type declarado.



Resumen del capítulo

- **RequestsLibrary** traduce estos conceptos directamente a keywords de Robot Framework, haciendo que la curva de aprendizaje sea mínima una vez dominados los fundamentos HTTP.
- El esquema **Bearer token** autentica al cliente incluyendo un token en el encabezado Authorization: Bearer de cada petición protegida.
- El flujo de autenticación tiene dos pasos: obtener el token en un endpoint dedicado y luego usarlo en las peticiones de negocio.
- Las **sesiones persistentes** en RequestsLibrary permiten configurar encabezados una sola vez (incluyendo el token) y reutilizarlos en todos los tests de la suite, reduciendo duplicación y mejorando el rendimiento.
- El ciclo de vida recomendado es: Suite Setup para crear la sesión y Suite Teardown con Delete All Sessions para liberarla.



Práctica 13: Suite de pruebas API con autenticación Bearer y validación JSON

Objetivo:

Consumir endpoints REST autenticados con token, validar estructura y valores de respuesta JSON.

Guía completa: [robfram-guide/Capitulo07/Practica13.md](#)



Práctica 14: Suite API data-driven: smoke y regresión desde CSV

Objetivo:

Construir una suite parametrizada desde CSV que cubra escenarios positivos y negativos con tags smoke/regresion.

Guía completa: [robfram-guide/Capitulo07/Practica14.md](#)



Referencias Bibliográficas

Capítulo 7: Automatización de APIs con RequestsLibrary

- ▶ <https://marketsquare.github.io/robotframework-requests/doc/RequestsLibrary.html>
- ▶ <https://www.rfc-editor.org/rfc/rfc9110>
- ▶ <https://developer.mozilla.org/es/docs/Web/HTTP/Status>
- ▶ <https://restfulapi.net/>
- ▶ <https://jsonplaceholder.typicode.com/>